Project 4: Out of Control Planning — Final Report

Darshon Singh (ds161) and Winson Lin (wl85) — COMP/ELEC/MECH 450/550

1. Problem statement

We plan dynamically feasible, collision-free motions for two systems whose motion obeys $\dot{q} = f(q,u)$ and that cannot move directly between nearby states: a torque-limited pendulum and a car-like vehicle in a street. Planning is done in OMPL's *control* framework: planners sample a control and a duration, integrate the ODE (RK4, 0.001 s step) to obtain the resulting state, and check every 0.02 s that the trajectory stays valid. We solve both problems with OMPL's control RRT and KPIECE1, implement the reachability-guided RRT (RG-RRT) of Shkolnik et al. (2009), and compare the three planners with OMPL's `Benchmark` class. Control durations are sampled from 1–25 propagation steps (0.02–0.50 s), and every planner uses goal bias 0.1. Because a control planner essentially never lands exactly on a goal state, each goal is a ball of the stated radius in the state-space metric.

2. Robots, state spaces, environments and queries

Pendulum. A unit bar rotating about one end, with no obstacles. State $q = (\theta, \omega) \in S^1 \times [-10, 10]$ (`SO2 × RealVector(1)`), $\theta = 0$ horizontal, dynamics $\dot{\theta} = \omega$, $\dot{\omega} = -9.81\cos\theta + \tau$, and control $\tau \in [-\tau_{\max}, \tau_{\max}]$ with $\tau_{\max} \in \{3, 5, 10\}$. The validity checker rejects states outside the bounds, i.e. $|\omega| > 10$; the angle is wrapped after propagation but the speed is never clamped. Query: from hanging at rest $(-\pi/2, 0)$ to upright at rest $(\pi/2, 0)$, goal radius 0.15 in $|\Delta\theta|_{S^1} + |\Delta\omega|$. KPIECE1 projection: (θ, ω), cells $(0.25, 0.5)$.

Car. A point vehicle (allowed by the handout) in a 12×10 m street with three closed building rectangles (x, y, w, h): $(3, 0, 1, 6.5)$, $(6, 3.5, 1, 6.5)$ and $(9, 0, 1, 6.5)$, so the car must weave north, south and north again through alternating gaps. State $q = (x, y, \theta, v) \in SE(2) \times [-2, 2]$ (reverse allowed), control $u = (\omega, \dot{v}) \in [-1, 1]^2$ (rad/s, m/s^2), dynamics $(\dot{x}, \dot{y}, \dot{\theta}, \dot{v}) = (v\cos\theta, v\sin\theta, \omega, \dot{v})$. The validity checker enforces the workspace and speed bounds and calls `isValidPoint` on (x, y); touching a building counts as a collision. Query: from $(1, 1, \pi/2, 0)$ (bottom left, facing north, at rest) to $(11, 9, \pi/2, 0)$ (top right, at rest), goal radius 0.5 in OMPL's metric $\|\Delta(x, y)\|_2 + 0.5|\Delta\theta| + |\Delta v|$. KPIECE1 projection: (x, y, v), cells $(0.5, 0.5, 0.5)$, so the projection tracks workspace progress and speed.

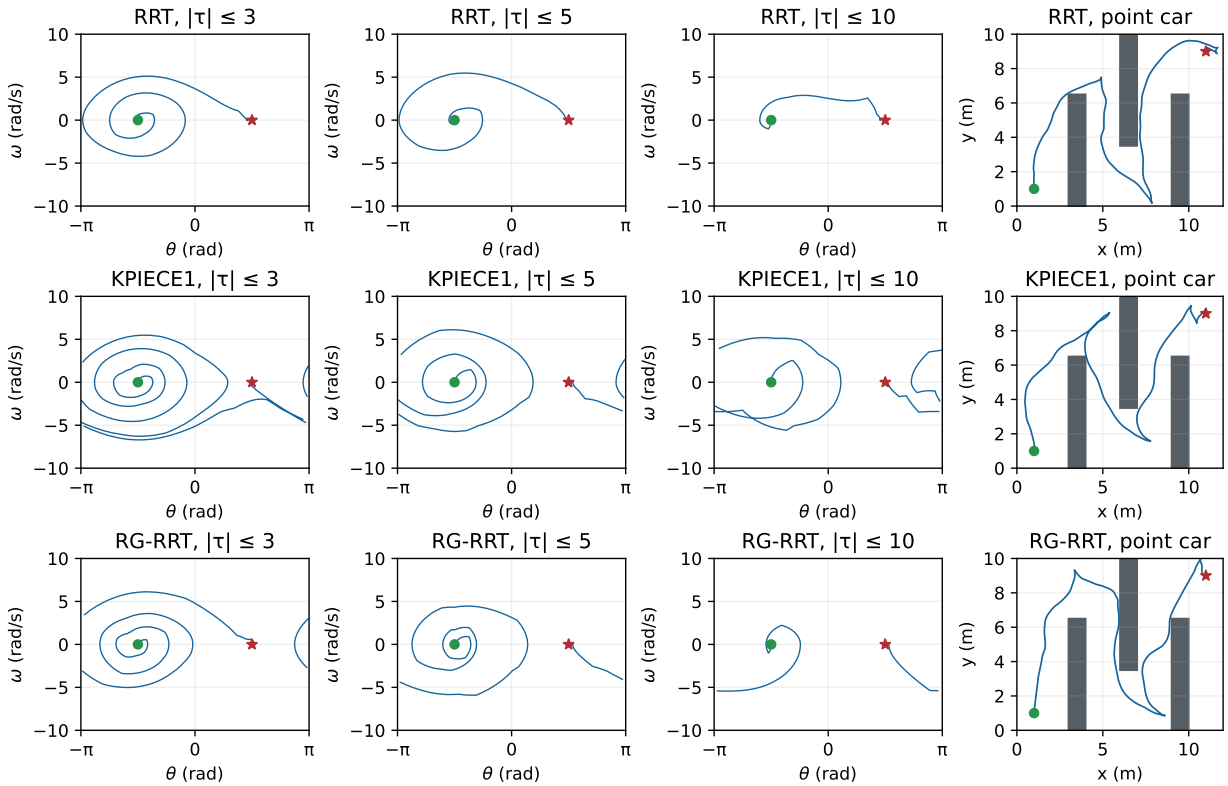


Figure 1. Example solutions (one run each, seed 450 except car KPIECE1, seed 451). Rows: RRT, KPIECE1, RG-RRT. Columns 1–3: pendulum phase-space paths for $\tau_{\max} = 3, 5, 10$; lines break where θ wraps at $\pm\pi$. Column 4: car position over the gray buildings. Green dot: start; red star: goal. All 12 paths were independently checked by re-integrating the saved controls with SciPy (RK45, samples $\leq$0.002 s

apart): every path stays within the bounds, avoids the buildings and ends inside the goal region; the largest per-segment endpoint error is below 4×10^{-11}.

3. RG-RRT implementation

`ompl::control::RGRRT` (`src/RG-RRT.{h,cpp}`) starts from OMPL's control RRT. Each `Motion` also stores $R(q)$, a vector of states. When a node is added, we apply each of k fixed controls for s propagation steps with `SpaceInformation::propagateWhileValid`. A reached state is stored only if the whole segment is valid, since an invalid state is not reachable. In each iteration we sample q_{rand} (goal bias 0.1) and find its nearest tree node q_{near}. If no state of $R(q_{\text{near}})$ is closer to q_{rand} than q_{near}, the sample is discarded and a new one is drawn. Otherwise q_{near} is extended towards q_{rand} with RRT's directed control sampler, and the new node gets its own reachable set. **Assumptions:** the control space is a bounded `RealVectorControlSpace`; the k controls are evenly spaced over the bounds of u_0 (torque for the pendulum, e.g. $\{-3, -2.4, \ldots, 3\}$ for $k = 11$, $\tau_{\max} = 3$; steering rate for the car), and the other control dimensions are held at the centre of their bounds (car acceleration $\dot{v} = 0$). Defaults: $k = 11$, $s = 5$ (0.1 s); both are planner parameters (`reachable_controls`, `reachable_steps`).

4. Pendulum: torque limits 3, 5 and 10

The torque bound decides whether the pendulum can be lifted directly. Holding the bar still at angle θ needs $\tau = 9.81 \cos\theta$, so a bound of $\tau_{\max}$ allows a static pose only where $|\cos\theta| \le \tau_{\max}/9.81$. With $\tau_{\max} = 10 > 9.81$ every angle can be held, and the planner can drive the bar almost straight up. With $\tau_{\max} = 5$ (static poses within 31° of vertical) and $\tau_{\max} = 3$ (within 18°) it cannot. The pendulum must instead *pump energy* by swinging back and forth. The energy $E = \frac{1}{2}\omega^2 + 9.81 \sin\theta$ must rise from -9.81 to $+9.81$, and $\dot{E} = \tau\omega$. A swing over about π rad adds at most about $\pi\tau_{\max}$, so at least $\lceil 19.62/(\pi\tau_{\max}) \rceil = 3, 2$ and 1 swings are needed for $\tau_{\max} = 3, 5, 10$. Figure 1 matches this. The $\tau_{\max} = 3$ paths spiral outward through several swings (7–8 reversals of ω for all three planners). The $\tau_{\max} = 5$ paths need 3–5 reversals, and the $\tau_{\max} = 10$ paths go up after 2–3. To check that this holds beyond single examples, we benchmarked all three planners at every torque bound (30 runs each, Table 1). For every planner the median trajectory duration falls steadily as the bound grows. RRT's falls from 7.4 s ($\tau_{\max} = 3$) to 4.9 s and then 3.4 s, and RG-RRT's from 8.5 to 5.7 to 3.3 s: fewer pumping swings are needed. Computation time hardly changes, since all runs succeed within about 2 s in this small 2-D state space. The torque limit changes the *shape and length of the motion* far more than the difficulty of finding it.

Table 1. Pendulum, all three planners at each torque bound (30 runs each, 30 s limit, 100% success).

Median over 30 runs	$\tau_{\max} = 3$	$\tau_{\max} = 5$	$\tau_{\max} = 10$
RRT path duration (s)	7.44	4.91	3.39
RRT computation time (s)	0.30	0.46	0.34
KPIECE1 path duration (s)	10.34	8.31	5.12
KPIECE1 computation time (s)	0.17	0.17	0.09
RG-RRT path duration (s)	8.54	5.72	3.32
RG-RRT computation time (s)	1.47	1.38	1.86

5. Benchmarks: RRT, KPIECE1 and RG-RRT

We used `ompl::tools::Benchmark` with torque 3 for the pendulum: 30 runs per planner with a 30 s limit for the pendulum, and 25 runs per planner with a 60 s limit for the car. Success means an exact solution inside the goal region. Path length in OMPL's control benchmarks is the trajectory duration, reported here for successful runs only. Tree nodes count all runs. RG-RRT uses $k = 11$ controls and $s = 5$ steps (0.1 s). Raw logs and `benchmark.db` (Planner Arena format) are in `results/benchmark/`. Runs from different experiments ran in parallel on an 8-core laptop, so absolute times include some CPU contention; planners within one experiment ran under the same load.

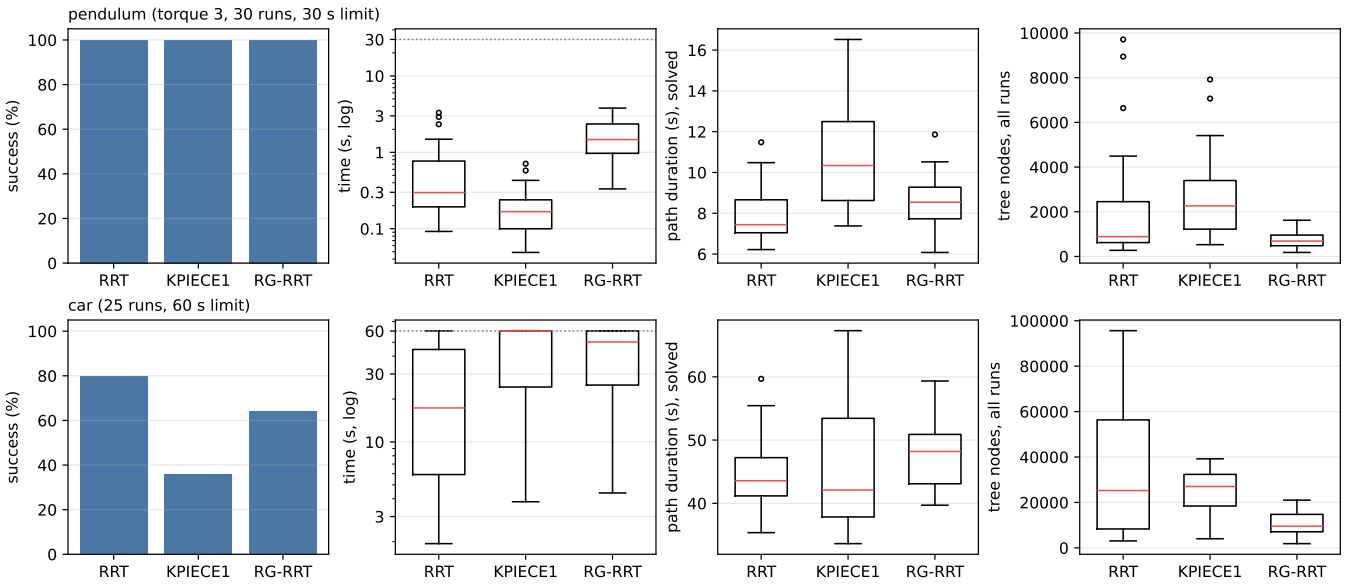


Figure 2. Benchmark of RRT, KPIECE1 and RG-RRT. Top: pendulum ($\tau_{\max} = 3$). Bottom: car. Columns: success rate, computation time (log scale; dotted line = time limit), trajectory duration of successful runs, and tree nodes.

Table 2. Benchmark summary. Duration: median over successful runs; tree nodes: median over all runs.

System / planner	runs	success	time (s): median, mean		duration (s)	tree nodes
Pendulum RRT	30	100%	0.30	0.67	7.44	888
Pendulum KPIECE1	30	100%	0.17	0.20	10.34	2262
Pendulum RG-RRT (11 ctrl, 5 steps)	30	100%	1.47	1.73	8.54	690
Car RRT	25	80%	17.32	25.03	43.56	25237
Car KPIECE1	25	36%	60.02	45.31	42.10	27029
Car RG-RRT (11 ctrl, 5 steps)	25	64%	50.28	41.33	48.20	9512

Pendulum. All three planners solve every run. KPIECE1 is fastest (median 0.17 s vs. 0.30 s for RRT and 1.47 s for RG-RRT): its projection is the full (θ, ω) state, so its coverage-driven search spreads quickly over the 2-D phase space. It does so with the most nodes (median 2262) and the longest trajectories (median 10.3 s, with more swings than needed). RRT finds the shortest trajectories (7.4 s). RG-RRT builds the smallest trees (690 nodes) and gives 8.5 s trajectories, but is the slowest in wall-clock time.

Car. The car is much harder. Its 4-D state has a speed dimension, and it must weave through three narrow passages. RRT is best here: 80% success with a median of 17 s. RG-RRT reaches 64% (median 50 s) and KPIECE1 only 36%; more than half of its runs hit the 60 s limit. KPIECE1's (x, y, v) grid has $24 \times 20 \times 8 \approx 3800$ cells, and its push for uniform coverage spends effort filling cells far from the route instead of pushing through the gaps. Trajectory durations of solved runs are similar for all three (medians 42–48 s); with 9–20 successful runs each, these differences are within the spread shown in Figure 2. RG-RRT again builds far fewer nodes (9.5k vs. 25k for RRT and 27k for KPIECE1).

6. Head-to-head: RRT vs. RG-RRT, and the cost of reachable sets

What reachability buys. In every experiment RG-RRT's trees are smaller than RRT's: on the pendulum 690 vs. 888 nodes in the main run and 2–3× fewer (727–1034 vs. 2218) in the study; on the car 9.5k vs. 25k. The rejection test works as intended. In our example runs it discarded about 75–80% of samples on the pendulum ($\tau_{\max} = 3, 5$) and about half on the car. Those samples lie in directions the node cannot move in, e.g. sideways for the car, or towards larger $|\omega|$ while gravity pulls the other way. RRT would have spent a propagation on each of them and added an unhelpful node.

What it costs. Every new node costs $k \cdot s$ extra propagation steps to build $R(q)$. With $k = 11$, $s = 5$ that is 55 steps, while an ordinary RRT extension samples 1–25 steps (13 on average), so a node is roughly 5× more expensive. Every discarded sample also costs a nearest-neighbour query and k distance evaluations. On our problems the smaller tree does not make up for this. RG-RRT with the default $(11, 5)$ is slower than RRT in wall-clock time on both systems (Table 2) and has lower success on the car within 60 s (64% vs. 80%). Path durations are similar.

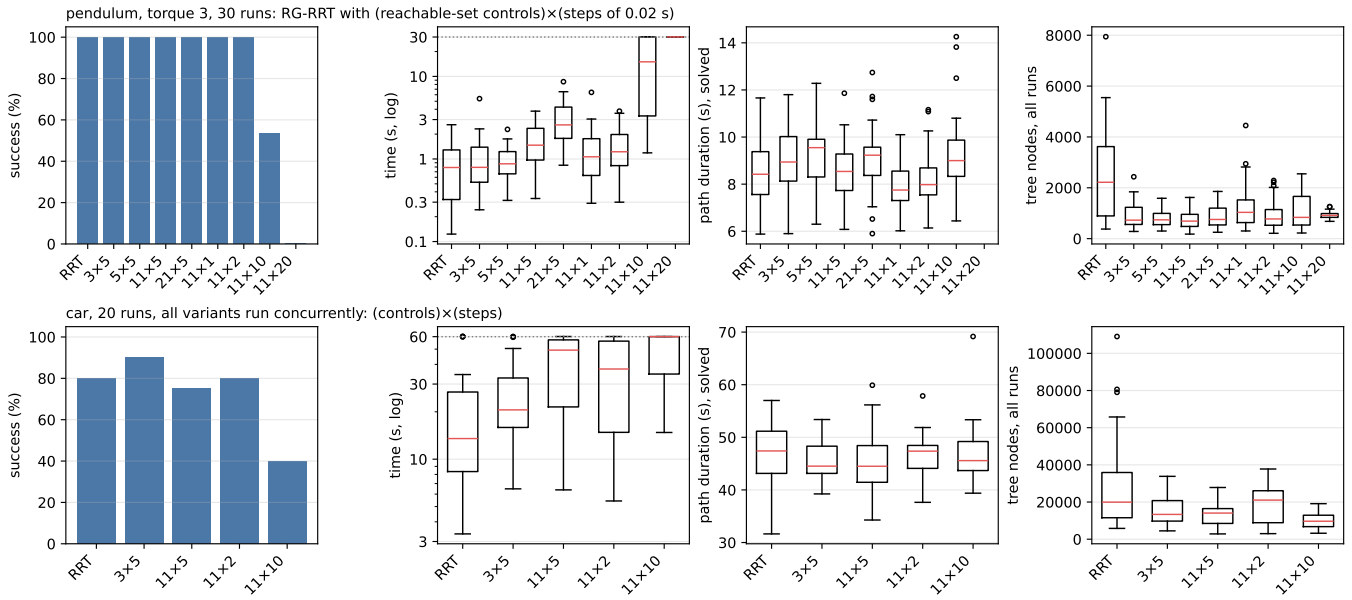


Figure 3. RG-RRT trade-off study. Labels are $k \times s$: number of reachable-set controls $\times$ duration in 0.02 s steps. Top: pendulum, $\tau_{\max} = 3$, 30 runs, 30 s limit; the 11×5 entry is the main experiment. Bottom: car, 20 runs, 60 s limit, with every variant (and RRT) run concurrently so they share the same CPU load.

Table 3. RG-RRT trade-off study (*main pendulum experiment).

System / planner	runs	success	time (s): median, mean		duration (s)	tree nodes
Pendulum RRT	30	100%	0.79	0.89	8.42	2218
Pendulum RG-RRT 3 ctrl, 5 steps	30	100%	0.79	1.06	8.94	727
Pendulum RG-RRT 5 ctrl, 5 steps	30	100%	0.87	0.94	9.55	740
Pendulum RG-RRT 11 ctrl, 5 steps*	30	100%	1.47	1.73	8.54	690
Pendulum RG-RRT 21 ctrl, 5 steps	30	100%	2.59	3.22	9.23	754
Pendulum RG-RRT 11 ctrl, 1 step	30	100%	1.06	1.42	7.75	1034
Pendulum RG-RRT 11 ctrl, 2 steps	30	100%	1.22	1.58	7.98	774
Pendulum RG-RRT 11 ctrl, 10 steps	30	53%	15.01	16.75	9.00	836
Pendulum RG-RRT 11 ctrl, 20 steps	30	0%	30.09	30.08	—	916
Car RRT	20	80%	13.52	22.53	47.41	19928
Car RG-RRT 3 ctrl, 5 steps	20	90%	20.57	27.37	44.52	13338
Car RG-RRT 11 ctrl, 5 steps	20	75%	49.28	39.97	44.50	14058
Car RG-RRT 11 ctrl, 2 steps	20	80%	37.39	35.12	47.37	21042
Car RG-RRT 11 ctrl, 10 steps	20	40%	60.01	46.80	45.57	9668

Number of controls k. Cost grows linearly with k, and the benefit saturates almost immediately. On the pendulum the tree size is nearly constant from $k = 3$ to $k = 21$ (727–754 nodes), while the median time rises from 0.79 s to 2.59 s. With a single control input, the extreme torques $\pm\tau_{\max}$ (both included when $k \geq 2$) already bound $R(q)$; extra intermediate controls rarely change the accept/reject decision. On the car, $k = 3$ (steering $-1, 0, 1$) was the best RG-RRT variant and the only one that beat RRT in success (90% vs. 80%). It was still slower (median 21 s vs. 14 s), and with 20 runs this success difference is within run-to-run variation. For example, the same 11×5 setting gave 64% in the main experiment and 75% here.

Length of the time period s. This is the more delicate parameter. Short periods (1–2 steps, 0.02–0.04 s) keep $R(q)$ close to q. RG-RRT then behaves more like RRT (more nodes, e.g. 1034 for 11×1), is cheap, and gives the shortest pendulum trajectories of any variant (7.75 s). Long periods hurt badly: 11×10 (0.2 s) solves only 53% of pendulum runs and 40% of car runs, and 11×20 (0.4 s) solves none. Each node is 2–4$\times$ more expensive. More importantly, the reachable states lie far from q: at $|\omega| \approx 6$ rad/s the pendulum moves over 2 rad in 0.4 s. Samples close to q_{near}, including goal samples, are then closer to q_{near} than to any reachable state and are discarded. The tree can no longer make the small corrections needed to enter the goal region. Long propagations can also leave the velocity bounds, which removes those states from $R(q)$. The time period should therefore be comparable to a single extension step, short enough that $R(q)$ describes local motion.

Verdict. RG-RRT clearly reduces wasted expansions and tree size. For the pendulum and car in this project, the fixed per-node cost of the reachable set outweighs that saving, so plain RRT is faster. Cheap reachable sets (few controls, short duration) are

the best compromise. RG-RRT should pay off more where nearest-neighbour search or collision checking dominates the cost of a short propagation, or where the metric is very misleading about reachability.

7. Difficulty, time spent and contributions

[Please fill in your own ratings and hours.] Exercise 1 (models, validity, RRT): difficulty **[?]**/10, **[?]** h. Exercise 2 (KPIECE1 and projections): **[?]**/10, **[?]** h. Exercise 3 (RG-RRT): **[?]**/10, **[?]** h. Exercise 4 (benchmarking and report): **[?]**/10, **[?]** h. Hardest part: **[e.g. choosing goal radii, propagation and reachable-set durations so that the planners solve reliably within the time limits, and getting RG-RRT's reachable-set bookkeeping right]**.

Contributions. Darshon: pendulum modeling, validity, RRT/KPIECE setup, and torque analysis. Winson: car modeling, street/collision checking, RRT/KPIECE setup, and visualization. **[RG-RRT and benchmarking split.]** Both shared integration, validation, interpretation and report writing.